

CSC144V Lab 3: Decision Making for Highway Driving with Deep Q-Learning

Algorithms and Systems for Autonomous Vehicles, Hofstra University, Fall 2026. Instructor: Zonghua Gu. Work in your groups, as in Lab 2. Due date: see the course website.

1. Overview

In Lab 1 you attacked a perception module and in Lab 2 you controlled a vehicle with a PID controller. In this lab you study decision making: a vehicle drives on a multi-lane highway among other cars, and has to decide when to speed up, slow down or change lanes. You will model the problem as a Markov decision process, write a hand-crafted baseline driver, train a Deep Q-Network (DQN) agent, and then modify the reward function by adding a penalty for changing lanes. The goal of the last part is to see for yourself how the reward you write down shapes the behaviour that the agent learns, including behaviour you did not intend.

What is new compared with the earlier version of this lab. The lab now uses the Gymnasium API with highway-env and Stable-Baselines3 (SB3) instead of the older gym + rl-agents combination. You do not need to fork the highway-env repository or edit library files: the lane-change penalty is added with a small wrapper class that is provided in the notebook. The task of studying a lane-change penalty, including the search for the point at which the agent prefers a collision to a lane change, is the same as before.

Prerequisites: L9.1 Markov Decision Process, L9.2 Q-Learning (DQN), and the Planning and Decision Making lectures. A GPU is not needed: the network is a small multilayer perceptron, so use the default CPU runtime of Colab.

2. Background

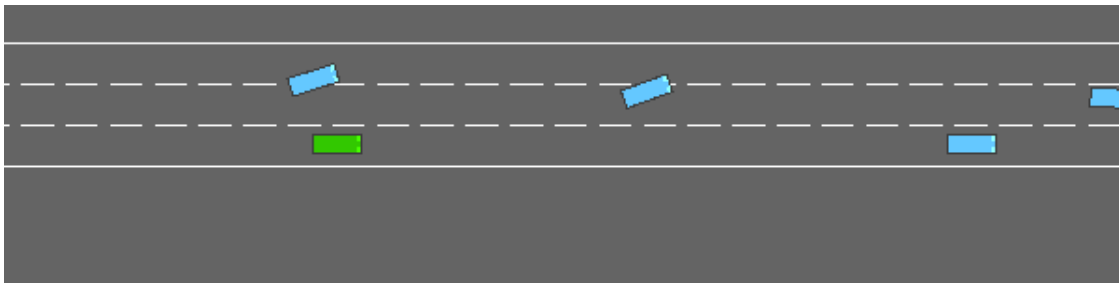


Figure 1. The simulator (highway-fast-v0). The green vehicle is the ego vehicle that you control; the blue vehicles are controlled by simple car-following and lane-change models.

highway-env (E. Leurent) simulates traffic with kinematic vehicle models. The environment highway-fast-v0 used here is a lighter variant of highway-v0: the same kind of vehicle and traffic models, but 3 lanes and 20 vehicles instead of 4 lanes and 50, and a coarser simulation step (5 Hz instead of 15 Hz), so it is roughly ten times faster, which is what makes training in the browser practical. The main elements of the MDP are:

Element	In highway-env
State (observation)	Kinematics: a 5 x 5 table with one row per vehicle (the ego vehicle and the 4 nearest others) and the columns [presence, x, y, vx, vy], normalised. Positions of the others are relative to the ego vehicle. The notebook provides <code>decode_obs</code> to convert it to metres and m/s.
Actions	5 discrete meta-actions: 0 LANE_LEFT, 1 IDLE, 2 LANE_RIGHT, 3 FASTER, 4

Element	In highway-env
	SLOWER. Low-level controllers (speed and steering, similar to the PID of Lab 2) turn them into acceleration and steering.
Reward	Weighted sum, normalised: collision_reward (-1 when crashed), right_lane_reward (0.1, larger in the rightmost lane), high_speed_reward (0.4, larger when the speed is in reward_speed_range = [20, 30] m/s), plus a term for staying on the road.
Episode	Ends at a collision or after 30 decision steps (one decision per second of simulated time).

The DQN agent uses the same hyperparameters as the dqn.json of the rl-agents library of the earlier version of this lab, translated to Stable-Baselines3 (two hidden layers of 256 units, gamma = 0.8, batch size 32, replay buffer of 15000 transitions, target network update every 50 steps, epsilon-greedy exploration with a final epsilon of 0.05). They are given in the notebook, and you should not need to change them, although you may wish to experiment in Task 3.

3. Setup

Open the notebook Lab3_Highway_Env.ipynb in Google Colab (Copy to Drive), and use the default CPU runtime. The first code cell installs highway-env and Stable-Baselines3 with pip and defines the helper functions:

```
%pip -q install highway-env stable-baselines3
import os; os.environ['SDL_VIDEODRIVER'] = 'offscreen' # headless rendering
env = make_env(config={}) # highway-fast-v0 with an
optional config dict
evaluate(policy, n_episodes=30, config=None) # crash rate, speed,
length, lane changes
rollout_gif(policy, 'episode.gif', config=None) # record and display one
episode
train_dqn(config=None, steps=15_000, lane_penalty=0.0) # returns (model, training
env)
```

- Training for 15000 steps takes about 10 minutes on the Colab CPU runtime. Task 4 needs several training runs, so start early, and keep the browser tab open (Colab disconnects idle sessions). For debugging, use steps = 2000 to 4000; results with so few steps are not meaningful.
- If a later cell complains about a package version after the installation, restart the runtime once (Runtime, Restart session) and run the cells again, starting from the first one.
- Only the last trained model of a run is kept in memory, but the notebook stores every agent in a Python dictionary, so you can evaluate all agents at the end. Save the GIFs and the tables as you go, because they are lost when the runtime is recycled.
- You may also run the notebook on your own machine (pip install highway-env stable-baselines3).

4. Task 1: Explore the environment (15 points)

- Run the cells that print the configuration, one observation and one step, and record a random driver as a GIF.
- Answer briefly in the notebook: (a) what are the state, action, reward and discount factor of this MDP, and is the observation Markov? (b) why does highway-env use meta-actions such as

LANE_LEFT and FASTER instead of raw steering and throttle, and what would change for the learning problem? (c) using `decode_obs`, how far ahead is the nearest vehicle in the ego lane at the start of the episode with seed 0? (d) what do the terms `collision_reward`, `right_lane_reward`, `high_speed_reward` and `on_road_reward` in `info['rewards']` measure, and how are they combined into the scalar reward?

5. Task 2: Rule-based baseline (15 points)

Before using reinforcement learning, write a simple hand-crafted driver, which is the benchmark that RL has to beat.

- Implement `rule_policy(obs, gap_m)`: decode the observation with `decode_obs`; if a vehicle is ahead in the ego lane ($dx > 0$ and $|dy| < 2$ m) and closer than `gap_m` metres, return SLOWER, otherwise return FASTER. (8 points)
- Evaluate the random driver, the always-IDLE driver, and your driver with three different values of `gap_m` on 30 episodes with fixed seeds, using `evaluate`. Show the results in one table (return, crash rate, mean speed, episode length, lane changes), and record one GIF of your best driver. (4 points)
- Answer: which gap threshold works best and why? How does the rule-based driver compare with the random and IDLE drivers? What are its limitations, that is, which situations does it handle badly? You may optionally add lane changes (check that the target lane is free). (3 points)

6. Task 3: Train a DQN agent (25 points)

- Train the DQN agent with the default reward for 15000 steps and plot the smoothed learning curve (mean return per training episode). (8 points)
- Evaluate the trained agent greedily on the same 30 fixed-seed episodes and add it to the table of Task 2. (6 points)
- Record a GIF of one test episode and describe what you see: speed, lane choice, how it handles slower cars. (4 points)
- Answer: does DQN beat your rule-based driver and the random driver? Why is the return a noisy training signal, and what does the learning curve tell you (is it still improving at the end)? What would you try to improve the agent (more steps, a different gamma, ...) and why? (7 points)

Note. Training and evaluation use different random seeds for the traffic, so the agent is tested on situations it has never seen. Results of a single run of 15000 steps vary from run to run (also with the seed of the agent); please do not expect every group to get the same numbers.

7. Task 4: Reward shaping with a lane-change penalty (40 points)

The default reward punishes collisions and rewards speed and driving in the right lane, but there is no cost for changing lanes. A lane change is not free in practice (comfort, risk, energy). The config of this version of `highway-env` contains an entry `lane_change_reward`, but it is not used by the highway reward: look at `info['rewards']` to convince yourself. You will add the missing term with the wrapper class `LaneChangePenalty(env, penalty)` provided in the notebook, which adds penalty (a negative number) to the reward whenever the ego vehicle starts a lane change. Use it through `train_dqn(config, lane_penalty=...)`. When you evaluate agents, always use the same environment without the penalty, so that the behaviour metrics are comparable.

7.1 Part A: warm-up in a trivial environment (10 points)

Use `config = dict(vehicles_count=0, lanes_count=2, initial_lane_id=0)`: there is no other traffic, there are two lanes, and the vehicle starts in the left lane. Train agents with `lane_penalty = 0` and at least two further values (for example -0.1 and -1), for 4000 to 6000 steps each (about 2 to 4 minutes).

- With no penalty, the vehicle should move to the right lane quickly and stay there. If it does not, the training was probably too short and the agent never tried the lane change action often enough.
- For each penalty report the lane in which the vehicle ends (use the helper `final_lane`) and the number of lane changes. Explain the change in behaviour by comparing the penalty with the discounted gain of the right-lane bonus: what is the smallest penalty at which staying in the left lane becomes optimal?

7.2 Part B: dense traffic (20 points)

Switch to a more crowded environment. We suggest `config = dict(vehicles_density=1.5, lanes_count=3)` (the default is 20 vehicles at density 1 on 3 lanes), in which a car-following policy alone is no longer enough to avoid every collision. In the default traffic, slowing down is often sufficient, so the penalty has little effect on the crash rate there. Train an agent for each of at least three values of `lane_penalty` (for example 0, -0.1, -1 and -10; it must be negative), 15000 steps each. For each agent report in one table the crash rate, the number of lane changes per episode, the mean speed and the episode length (30 fixed-seed episodes), and record one GIF.

- Make the penalty more and more negative until you see the vehicle prefer a collision to a lane change that would have avoided it. Describe what you see in the GIFs.
- Your report must include a pair of configurations that differ only in the lane-change penalty: one with no or rare collisions, and one with frequent collisions. Include screenshots (GIF frames) and a description of the observed behaviour of each. If you cannot find such a pair, show the evidence you gathered (for example more evaluation episodes, more traffic, more training steps) and explain why.
- Be careful with statistics: 30 episodes and one training seed give a noisy estimate of a crash rate, and a difference of 10 percentage points may be noise. A good report states how much confidence it has in each comparison, and if you have the time, repeats the critical pair with a second seed (the `seed` parameter of `train_dqn`).

7.3 Part C: one more reward change (10 points)

Choose one other modification of the reward configuration and train an agent in the dense environment. Suggestions: `safe` (`collision_reward` -5, `high_speed_reward` 0.1), `fast` (`high_speed_reward` 1.0), `keep right` (`right_lane_reward` 1.0). Write down what you expect before you train, then compare the metrics with the agent of Part B with no penalty, and explain whether the behaviour changed as expected.

7.4 Discussion (to be answered in the notebook or in the report)

- Why is it wrong to compare the returns of agents trained with different rewards, and what do you compare instead? (hint: look at the table of behaviour metrics)
- Name one way in which an agent could exploit your reward (reward hacking), or explain why a change of the reward had little effect.
- Why does a very large lane-change penalty not simply make the agent brake? What does braking cost in this reward, and what else is on the road?

- Design a reward for a safe but comfortable driver (few lane changes, no collisions) and say which weights you would tune and how you would check the result.

8. Submission

- A Colab link (sharing set to anyone with the link) or a Lab3-GroupName.ipynb file, executed so that all outputs, learning curves, tables and GIFs are visible. If you use Colab, send the link. Otherwise submit the notebook to Canvas.
- A short report in PDF (at most 6 pages), or text blocks inside the notebook, with the tables, learning curves, GIF frames (screenshots) and the written answers of Tasks 1 to 4, and in particular a description of each configuration you tried and your observations. Because only the agents in memory can be displayed in the notebook, the screenshots and descriptions are essential for the configurations you trained earlier.
- Use at most one submission per group, with the names of all members on the first page.

9. Grading

Part	Points
Task 1: explore the environment, short answers	15
Task 2: rule-based driver (8), comparison table and GIF (4), answers (3)	15
Task 3: training curve (8), evaluation (6), GIF (4), answers (7)	25
Task 4: warm-up (10), dense traffic and the pair of configurations (20), one more reward change (10)	40
Clarity of the report and code	5
Total	100

Hints. (1) More complex environments need more training, so keep the environment only as complex as the question demands. (2) You may increase `vehicles_density`, or change `reward_speed_range`, to increase the chance of collisions. (3) A learning curve that is still rising at the end of training means that you should train longer before drawing conclusions. (4) The notebook uses fixed evaluation seeds, so two policies are tested on exactly the same traffic situations.

References

- E. Leurent, An Environment for Autonomous Driving Decision-Making, 2018. Code: github.com/Farama-Foundation/HighwayEnv (documentation: highway-env.farama.org).
- A. Raffin et al., Stable-Baselines3: Reliable Reinforcement Learning Implementations, JMLR 2021. Documentation: stable-baselines3.readthedocs.io.
- V. Mnih et al., Human-level control through deep reinforcement learning, Nature 2015 (DQN).
- Lecture notes L9.1 Markov Decision Process and L9.2 Q-Learning, and the slides Lab3 Highway Env.